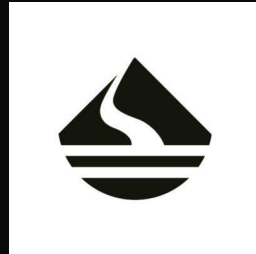




Security Assessment Final Report



Alluvial SMS Lite

September 2025

Prepared for Alluvial

[Table of Contents](#)

Project Summary	3
Project Scope	3
Project Overview	3
Privileged Roles & Actors	4
Findings Summary	5
Severity Matrix	5
Detailed Findings	6
Medium Severity Issues	8
M-01: Outgoing beneficiary can frontrun replacement to sweep accumulated funds	8
Low Severity Issues	10
L-01: Staker can bypass transfer restrictions by frontrunning	10
L-02: Deployment access control risk	11
L-03: TVS map can be desynchronized on transfer	12
L-04: Missing validation for ConfigManager.setMinSweepAmount()	13
L-05: Existing validator sets become unusable after upgrade due to missing TVSMap migration	14
Informational Severity Issues	15
I-01: Deposit contract address set in a constructor has no effect for proxy	15
I-02: Paused state blocks admin recovery operations	16
I-03: Missing __ReentrancyGuardTransient_init() in orchestrator init	17
I-04: BatchTVSTransfer exposes unprotected transferTVSs() function that could be misused	18
I-05: Inconsistent version string format between TVS and other contracts	19
I-06: Late validation of msg.value in TVS consolidate() function may lead to unnecessarily wasted gas...	20
About Certora	21

Project Summary

Project Scope

Repository Name	Platform	Commit Reviewed	Commit Fixed
sms-lite	EVM	c6db90d3	77ab2856
tvs	EVM	f5c73298	6d4345be

Project Overview

This document describes the security review of Alluvial SMS Lite. The work was undertaken from **September 4th, 2025** to **September 11th, 2025**.

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

The system includes:

- Upgradeable Orchestrator (`OrchestratorUpgradeableBase`, `OrchestratorV1`) using UUPS.
- Factory (`OrchestratorFactory`) to deploy new orchestrators and a TVS Beacon per instance.
- Batch helper contracts: TVS creation (`BatchUpgradeableTVSCreator`), TVS transfer (`BatchTVSTransfer`), deposit to consensus layer (`BatchDeposit`), plus other batch utilities.
- State is persisted via unstructured storage libraries.

About SMS Lite & TVS

- **`OrchestratorV1.sol`** is a UUPS-upgradeable orchestrator that manages validator sets (TVSs) via a per-instance beacon, config/state (beneficiary, config manager, deposit contract), access roles, pausing, and batch operations; it can create TVSs as beacon proxies and exposes role-gated management functions.
- **`OrchestratorFactory.sol`** is a UUPS-upgradeable factory that stores implementation addresses, deploys an `UpgradeableBeacon` per orchestrator, instantiates the

orchestrator via ERC1967Proxy with initialization data, transfers beacon ownership to the orchestrator, tracks instance IDs, and lets admins update implementations.

- **TVS.sol** is an abstract UUPS-upgradeable implementation of the Transferable Validator Set (TVS) that interacts with Pectra execution layer contracts for partial withdrawals and consolidations.

Privileged Roles & Actors

- **OrchestratorV1**
 - **DEFAULT_ADMIN_ROLE** – controls upgrades, unpause, beacon impl, config manager address, and owner changes.
 - **VALIDATOR_SET_CREATOR_ROLE** – creates TVSs;
 - **CONFIG_MANAGER_ROLE** – updates SMS config via delegatecall to ConfigManager
 - **VALIDATOR_MANAGER_ROLE** – requests exits/withdraws;
 - **BENEFICIARY_ROLE** – sets/syncs beneficiary;
 - **VALIDATOR_SET_TRANSFER_ROLE** – transfers/sweeps TVSs;
 - **DEPOSIT_CONSENSUS_LAYER_ROLE** – deposits validators;
 - **ETH_WITHDRAWER_ROLE** – withdraws ETH;
 - **DEPOSITOR_ROLE** – deposits ETH;
 - **PAUSER_ROLE** – pauses;
 - **TRANSFER_TO_LIST_ROLE** – toggles transfer gating;
 - **VALIDATOR_SET_RECIPIENT_ROLE** – required as recipient when gating. Owner is granted all; optional staker/keeper get predefined subsets at init.
- **OrchestratorFactory**
 - **DEFAULT_ADMIN_ROLE** – updates implementation addresses and authorizes upgrades;
 - **ORCHESTRATOR_DEPLOYER_ROLE** – deploys orchestrators and their per-instance beacons. By default, the deployer is granted both roles at initialization unless reassigned.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	0	–	–
High	0	–	–
Medium	1	1	1
Low	5	5	3
Informational	6	6	6
Total	12	12	10

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

Issue ID	Issue Title	Severity	Status
M-01	Outgoing beneficiary can frontrun replacement to sweep accumulated funds	Medium	Fixed
L-01	Staker can bypass transfer restrictions by frontrunning	Low	Acknowledged
L-02	Deployment access control risk	Low	Fixed
L-03	TVS map can be desynchronized on transfer	Low	Fixed
L-04	Missing validation for ConfigManager.setMinSweepAmount()	Low	Fixed
L-05	Existing validator sets become unusable after upgrade due to missing TVSMap migration	Low	Acknowledged
I-01	Deposit contract address set in a constructor (no effect for proxy)	Informational	Fixed
I-02	Paused state blocks admin recovery operations	Informational	Fixed
I-03	Missing __ReentrancyGuardTransient_init in orchestrator init	Informational	Fixed

I-04	BatchTVSTransfer exposes unprotected transferTVSs() function that could be misused	Informational	Fixed
I-05	Inconsistent version string format between TVS and other contracts	Informational	Fixed
I-06	Late validation of msg.value in TVS consolidate() function may lead to unnecessarily wasted gas	Informational	Fixed

Medium Severity Issues

M-01: Outgoing beneficiary can frontrun replacement to sweep accumulated funds

Severity: Medium	Impact: High	Likelihood: Low
BatchSweep.sol#L18-L23	Fix PR: fix: #132 [Medium] Outgoing beneficiary can frontrun replacement to sweep accumulated funds	Status: Fixed

Description: The `sweepToBeneficiary()` function in `OrchestratorV1` is permissionless and allows anyone to trigger fund sweeps from TVS contracts to their configured beneficiary addresses. When combined with `setBeneficiaryAndSyncToTVSs()`, this creates a front-running vulnerability where an outgoing beneficiary can monitor the mempool and sweep accumulated funds before being replaced. Thus, an outgoing beneficiary can claim funds that were intended for their replacement, resulting in direct financial loss. While the actor must be the current beneficiary (a previously trusted party), the vulnerability manifests precisely when that trust relationship is being terminated.

Attack scenario:

1. Alice is the current beneficiary across multiple TVS contracts
2. TVS contracts accumulate 20 ETH over time
3. Admin calls `setBeneficiaryAndSyncToTVSs(Bob, [tvs1, tvs2, tvs3])` to replace Alice with Bob
4. Alice monitors mempool and sees the pending transaction
5. Alice frontruns with `sweepToBeneficiary([tvs1, tvs2, tvs3], [10e18, 6e18, 4e18])` using higher gas
6. Alice receives all 20 ETH as she is still the active beneficiary
7. Bob's beneficiary update succeeds but accumulated TVS contracts are now swept

Recommendation: Add access control to the `sweepToBeneficiary()` function:

TypeScript

```
function sweepToBeneficiary(  
    address[] calldata sources,  
    uint256[] calldata amounts  
) external  
    onlyRole(SWEEPER_ROLE) // Add this  
    isNotPaused  
{  
    _sweepToBeneficiary(sources, amounts);  
}
```

This follows established DeFi patterns where sweep functions require authorized roles, preventing unauthorized or adversarial fund extractions during beneficiary transitions.

Customer response: Fixed.

Fix review: The fix looks good.

Low Severity Issues

L-01: Staker can bypass transfer restrictions by frontrunning

Severity: Low	Impact: Low	Likelihood: Low
OrchestratorBase.sol#L322-L326	-	Status: Acknowledged

Description: When the Owner enables transfer restrictions via the TransferToList feature, a Staker can frontrun this transaction and transfer validator sets to arbitrary recipients before the restriction takes effect.

In practice, the likelihood is very low. The Staker is a semi-trusted role and the frontrunning only accelerates actions they could already perform. Exploiting this requires adversarial behavior, mempool monitoring and precise timing – all in a scenario where trust in the Staker is already compromised.

Recommendation: If stronger guarantees are desired, implement a timelocked 2-step process for transferring validator sets. The VALIDATOR_SET_RECIPIENT_ROLE should be validated during the confirmation step.

Customer response: Acknowledged, Whilst we acknowledge the potential risk, for those with a higher risk aversion, we recommend always keeping the transferToListEnabled feature active when deploying the Orchestrator. All roles (staker, beneficiary, and owner) will remain under the same customer. The only time that we will advise people to turn off transferToListEnabled is if they are using a custodian to protect TVS transferTo destinations.

L-02: Deployment access control risk

Severity: Low	Impact: Low	Likelihood: Low
OrchestratorFactory.sol#L48-L49 OrchestratorV1.sol#L28-L51	Fix PR: [fix] #122 Deployment access control risk	Status: Fixed

Description: OrchestratorFactory grants the deployer (`msg.sender`). If you forgot to revoke the deployer in the deployment script the deployer will keep admin access to the factory.

Additionally, `tvSBeacon` is passed in the `OrchestratorV1.initialize()` function. The deployment script must transfer the Beacon ownership to the Orchestrator. It is easy to miss it in the deployment script. If this step is missed, the beacon proxy will remain upgradeable by the deployer which is critical.

Recommendation:

- Just like in `OrchestratorV1`, pass an owner and grant the roles to this owner instead of `msg.sender`.
- Assert that `TVSBeaconAddress.get()` is owned by `address(this)` in `OrchestratorV1.initialize()`.

Customer response: Fixed, We are confident in our decision not to implement the second recommendation (to add a `TVSBeacon` owner check to `OrchestratorV1`), since its deployment is exclusively intended through the `OrchestratorFactory`.

Fix review: The fix looks good. It's OK not to fix the second recommendation if you only plan to deploy through the factory.

L-03: TVS map can be desynchronized on transfer

Severity: Low	Impact: Low	Likelihood: Low
BatchTVSTransfer.sol#L43-L64	Fix PR: [fix] #121 TVS map can be desynchronized on transfer [PE-3233]	Status: Fixed

Description: BatchTVSTransfer._transferTVSs() sets TVSMap entry to false. If the _newOwner remains the orchestrator, the entry state is incorrectly left to false despite no effective change in ownership, breaking invariants that rely on TVSMap to validate ownership.

Recommendation: Revert if the new owner is address(this).

Customer response: Fixed

Fix review: The fix looks good.

L-04: Missing validation for ConfigManager.setMinSweepAmount()

Severity: Low	Impact: Low	Likelihood: Low
MinSweepAmount.sol#L22-L28	Fix PR: [fix] #117 Missing validation for ConfigManager.setMinSweepAmount() [PE-3233]	Status: Fixed

Description: The NatSpec for `set()` in `MinSweepAmount` says that the function should revert if the new value is invalid and must be a multiple of 32 ETH.

However, the implementation performs no validation checks. This can lead to inconsistent or unintended configuration values being stored (e.g., 0 or non-multiples of 32 ETH) which may cause downstream logic to behave incorrectly.

Recommendation: Add input validation to ensure that `_newValue` is greater than zero and a multiple of 32 ETH, reverting otherwise.

Customer response: Fixed

Fix review: The fix looks good.

L-05: Existing validator sets become unusable after upgrade due to missing TVSMap migration

Severity: Low	Impact: Low	Likelihood: Low
MinSweepAmount.sol#L22-L28	-	Status: Acknowledged

Description: The TVSMap upgrade introduces a breaking change that prevents deposits to validator sets created before the upgrade.

The updated `depositValidatorSetToConsensusLayer()` function checks if a validator set belongs to the orchestrator by looking it up in TVSMap. If the address is not registered, it reverts with `UnknownTVS()`. However, validator sets created before the upgrade will not exist in the map since it is a new storage mapping. There is no migration logic to populate TVSMap with pre-existing validator sets.

Recommendation: Add a migration function to register existing validator sets after upgrade:

```
TypeScript
function migrateExistingTVS(address[] calldata existingValidatorSets, address owner) external
onlyRole(DEFAULT_ADMIN_ROLE) {
    for (uint256 i = 0; i < validatorSets.length; i++) {
        // Perform ownership or other checks if needed
        require(ITVS(validatorSets[i]).owner() == address(this), "Not our TVS");
        TVSMap.set(validatorSets[i], true);
    }
    emit TVSMigrationCompleted(validatorSets.length);
}
```

Execute this function as part of the upgrade process to ensure continuity of operations.

Customer response: Acknowledged, We plan to deploy fresh instances of the contracts after completion of the audit, rather than migrating from an older version. Therefore, a migration function will not be necessary.

Informational Severity Issues

I-01: Deposit contract address set in a constructor has no effect for proxy

Severity: Informational	Impact: Low	Likelihood: Low
BatchDeposit.sol#L66-L68	Fix PR: [fix] #125 Deposit contract address set in a constructor (no effect for proxy)	Status: Fixed

Description: Setting DepositContractAddress inside the BatchDeposit constructor writes to the implementation's storage, not the proxy's storage. With UUPS, this initialization has no effect at runtime.

Recommendations: Keep the constructor empty and initialize the deposit address from the proxy's initializer path (already done in `_saveBaseConfig`).

Customer response: Fixed

Fix review: The fix looks good.

I-02: Paused state blocks admin recovery operations

Severity: Informational	Impact: Low	Likelihood: Low
OrchestratorBase.sol#L573-L586	Fix PR: [fix] Paused state blocks admin recovery operations	Status: Fixed

Description: Admin functions `grantRole()` and `revokeRole()` include the `isNotPaused` modifier, preventing role adjustments during an incident.

Recommendations: Remove `isNotPaused` from admin-only role management.

Customer response: Fixed.

Fix review: The fix looks good.

I-03: Missing `__ReentrancyGuardTransient_init()` in orchestrator init

Severity: Informational	Impact: Low	Likelihood: Low
OrchestratorV1.sol#L28-L51	Fix PR: fix: added initialization for reentrancy guard (#128)	Status: Fixed

Description: The orchestrator inherits `ReentrancyGuardTransientUpgradeable`, but the initializer is not invoked. `__ReentrancyGuardTransient_init()` does nothing; it's just a convention to include it in the initializer.

Recommendations: Call `__ReentrancyGuardTransient_init()` in `initialize()`.

Customer response: Fixed.

Fix review: The fix looks good.

I-04: BatchTVSTransfer exposes unprotected transferTVSs() function that could be misused

Severity: Informational	Impact: Low	Likelihood: Low
BatchTVSTransfer.sol#L20-L35	Fix PR: [fix] #120 BatchTVSTransfer exposes unprotected transferTVSs() function that could be misused	Status: Fixed

Description: The BatchTVSTransfer contract implements an external transferTVSs() function without access control, relying on OrchestratorBase to override it with security checks.

While the current system is secure (OrchestratorBase properly overrides with role checks and TransferToListEnabled protection), the base contract exposes an unprotected external function. If a developer mistakenly inherits directly from BatchTVSTransfer instead of OrchestratorBase, they would get unrestricted TVS transfer capabilities without any permission checks or transfer allowlist enforcement.

This design pattern creates unnecessary risk for future development or code forks. Although it requires a development error to exploit, following secure-by-default principles would prevent this possibility entirely.

Customer response: Fixed.

Fix review: The fix looks good.

I-05: Inconsistent version string format between TVS and other contracts

Severity: Informational	Impact: Low	Likelihood: Low
TVSUpgradeable.sol#L96-L99 TVSImmutableBase.sol#L33-L36	Fix PR: [fix] #52 Inconsistent version string format between TVS and other contracts (#53)	Status: Fixed

Description: The TVS contracts use a different version string format compared to other contracts in the project. TVS contracts prefix their version strings with "v" (e.g. "v1.0.0 U" and "v1.0.0 I") while other contracts like `OrchestratorBase` and `OrchestratorFactory` return version strings without the "v" prefix (e.g. "1.2.0").

This inconsistency in formatting conventions reduces code uniformity across the codebase and could lead to integration issues for external systems or off-chain components that parse version strings from the `version()` function, potentially requiring special handling for different contract types.

Customer response: Fixed.

Fix review: The fix looks good.

I-06: Late validation of msg.value in TVS consolidate() function may lead to unnecessarily wasted gas

Severity: Informational	Impact: Low	Likelihood: Low
TVS.sol#L127-L128	Fix PR: fix: #51 Late validation of msg.value in TVS.consolidate() function may lead to unnecessarily wasted gas (#54)	Status: Fixed

Description: The `consolidate()` function validates whether `msg.value` covers fees only after processing all consolidation requests. This wastes gas when transactions fail due to insufficient funds.

Currently, the function executes all consolidation operations (making external calls and paying fees) before checking if `msg.value` was sufficient. In contrast, `withdraw()` validates fees upfront before doing any work.

Recommendations: Validate if `msg.value` is sufficient to cover consolidation fees before processing any requests.

Customer response: Fixed.

Fix review: The fix looks good.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.